

实验一：进程间同步/互斥问题

陈天乐 无 25

2025 年 6 月 3 日

目录

1	实验目的	2
2	实验题目：银行柜员服务问题	2
2.1	问题描述	2
2.2	实现要求	2
2.3	输出要求	2
2.4	程序设计思路	2
2.4.1	顾客类定义	2
2.4.2	全局信号量	3
2.4.3	顾客到达处理函数	3
2.4.4	柜员处理函数	4
2.4.5	主函数	5
2.5	实验测例	6
3	实验结果	6
4	思考题	7
5	实验总结	8
6	源代码	9

1 实验目的

1. 通过对进程间通信同步/互斥问题的编程实现，加深理解信号量和 P、V 操作的原理；
2. 对 Windows 或 Linux 涉及的几种互斥、同步机制有更进一步的了解；
3. 熟悉 Windows 或 Linux 中定义的与互斥、同步有关的函数。

2 实验题目：银行柜员服务问题

本实验选择了银行柜员服务问题，操作系统平台选为 Windows，实现语言为 C++。

2.1 问题描述

银行有 n 个柜员负责为顾客服务，顾客进入银行先取一个号码，然后等着叫号。当某个柜员空闲下来，就叫下一个号。

编程实现该问题，用 P、V 操作实现柜员和顾客的同步。

2.2 实现要求

1. 某个号码只能由一名顾客取得；
2. 不能有多于一个柜员叫同一个号；
3. 有顾客的时候，柜员才叫号；
4. 无柜员空闲的时候，顾客需要等待；
5. 无顾客的时候，柜员需要等待。

2.3 输出要求

对于每个顾客需输出进入银行的时间、开始服务的时间、离开银行的时间和服务柜员号。

2.4 程序设计思路

首先，将顾客封装成一个 Customer 类，这个类中会有顾客的序号、拿到的号、到达时间、服务时间、离开时间等信息。然后对于问题要求，因为一个号只可以由一个顾客取得，因此需要一个信号量用于管理取号操作；一个柜员叫一个号，因此设定一个信号量管理叫号，只允许有一个柜员叫这个顾客的号；设定一个到达顾客队列的统计信号量，当该信号量为 0 时柜员需要等待；设定一个空闲柜员的统计信号量，当无空闲柜员时，到达的顾客需要等待。

由于需要输出入银行的时间、开始服务的时间、离开银行的时间和服务柜员号，需要设计相关函数，记录顾客到达银行的时间（这个一般是输入数据给出），然后根据等待时间来计算开始服务的时间和离开银行的时间。以下是代码的分块解读。

2.4.1 顾客类定义

```
1 class Customer {
2 private:
3     int id;           // 顾客序号
4     int num;          // 取号
5     int enter_time;   // 进入银行时间
```

```

6  int service_time;           // 需要服务的时间
7  int start_time;            // 开始服务时间
8  int leave_time;           // 离开银行时间
9  int server_id;            // 服务柜员号
10
11 public:
12  Customer(int id = 0, int enter_time = 0, int service_time = 0)
13      : id(id), enter_time(enter_time), service_time(service_time),
14        num(-1), start_time(0), leave_time(0), server_id(0) {}
15
16  //private信息接口
17  int get_id() const { return id; }
18  int get_num() const { return num; }
19  int get_enter_time() const { return enter_time; }
20  int get_service_time() const { return service_time; }
21  int get_start_time() const { return start_time; }
22  int get_leave_time() const { return leave_time; }
23  int get_server_id() const { return server_id; }
24
25
26  //设置服务接口
27  void set_id(int idx) { id = idx; }
28  void set_num(int number) { num = number; }
29  void set_enter_time(int t) { enter_time = t; }
30  void set_service_time(int t) { service_time = t; }
31  void set_start_time(int t) { start_time = t; }
32  void set_leave_time(int t) { leave_time = t; }
33  void set_server_id(int id) { server_id = id; }
34 };

```

这一部分代码定义了顾客类，该类中有所有顾客的序号、时间信息。

2.4.2 全局信号量

```

1  // 全局共享资源（保持不变）
2  int current_num = 0;           // 当前最大取号数（用于生成唯一号码）
3  std::mutex mtx_num;           // 保护取号操作的互斥锁
4  std::queue<Customer> wait_queue; // 等待服务的顾客队列
5  std::mutex mtx_queue;         // 保护队列操作的互斥锁
6
7  // 同步变量
8  int customer_num = 0;         // 等待服务的顾客数量（替代customer_waiting信号量）
9  std::condition_variable cv_customer; // 顾客到达的条件变量（通知柜员）
10 int available_servers;         // 可用柜员数量
11 std::condition_variable cv_server; // 柜员空闲的条件变量（通知顾客）

```

这一部分定义了信号量，用于做互斥操作和队列同步操作。

2.4.3 顾客到达处理函数

```

1  // 顾客到达事件处理函数
2  void customer_enter(Customer& customer) {
3      // 取号（互斥操作，确保号码唯一）
4      {
5          std::lock_guard<std::mutex> lock(mtx_num);
6          customer.set_num(++current_num);
7      }
8
9      // 等待可用柜员（无空闲柜员时阻塞）
10 {

```

```

11     std::unique_lock<std::mutex> lock(mtx_queue);
12     cv_server.wait(lock, [&] { return available_servers > 0; }); // 等待柜员空闲
13     available_servers--;
14 }
15
16 // 加入等待队列（互斥操作）
17 {
18     std::lock_guard<std::mutex> lock(mtx_queue);
19     wait_queue.push(customer);
20     customer_num++;
21 }
22
23 // 通知有新顾客等待
24 cv_customer.notify_one(); // 唤醒一个柜员
25 }

```

这一部分定义的函数是负责给到达的顾客进行取号、判断是否有空闲柜员并唤醒空闲柜员的。在取号时先将 `mtx_num` 信号量锁住，然后取号；然后在加入队列时先将 `mtx_queue` 锁住，如果没有空闲柜员则需要 `wait` 等到空闲柜员数量大于 0，如果有则直接占用一个空闲柜员；如果队列是空闲的，那么该顾客会唤醒一个柜员。

2.4.4 柜员处理函数

```

1     void server(int server_id) {
2         thread_local int last_leave_time = 0; // 线程局部变量，记录该柜员上次服务结束时间
3
4         while (true) {
5             // 等待有顾客需要服务
6             Customer customer;
7             {
8                 std::unique_lock<std::mutex> lock(mtx_queue);
9                 cv_customer.wait(lock, [&] { return customer_num > 0; }); // 等待顾客到达，无顾客时阻塞
10                customer = wait_queue.front();
11                wait_queue.pop();
12                customer_num--;
13            }
14
15            // 计算服务时间
16            int start_time = customer.get_enter_time() > last_leave_time ? customer.get_enter_time() :
17                last_leave_time;
18            int leave_time = start_time + customer.get_service_time();
19
20            // 记录服务信息
21            customer.set_start_time(start_time);
22            customer.set_leave_time(leave_time);
23            customer.set_server_id(server_id);
24
25            // 输出结果
26            cout << "-----" << endl;
27            cout << "The " << customer.get_id() << " customer come! His requiring service time is: "
28                << customer.get_service_time() << endl;
29            cout << "The enter time: " << customer.get_enter_time() << endl;
30            cout << "The service start time: " << customer.get_start_time() << endl;
31            cout << "The leave time: " << customer.get_leave_time() << endl;
32            cout << "The server's id: " << customer.get_server_id() << endl;
33
34            // 模拟服务耗时
35            std::this_thread::sleep_for(std::chrono::seconds(customer.get_service_time()));
36
37            // 服务完成，释放柜员
38        }
39    }

```

```

36 {
37     std::lock_guard<std::mutex> lock(mtx_queue);
38     available_servers++;
39 }
40 cv_server.notify_one(); // 通知顾客有柜员空闲
41
42 // 更新柜员空闲时间
43 last_leave_time = leave_time;
44 }
45 }

```

这一部分定义的函数，是柜员用于叫号、等待顾客到达、服务顾客并在服务结束后输出该名顾客的信息。在某个柜员准备叫号时，先将 `mtx_queue` 信号量锁住，如果队列没有人那么就一直等待直到顾客到来，如果队列有人则将服务队首的顾客，等待队列的数量-1；依次计算完开始服务时间和顾客离开时间，输出结果，在服务过程中使用 `sleep` 让该线程休眠，模拟服务用时；在服务结束后会通知顾客有空闲柜员产生。

2.4.5 主函数

```

1 int main() {
2     int server_num = 2; // 柜员数量
3     string test_data_file = "C:/Users/skyhappy/Desktop/test.txt";
4
5
6     // 从文件中读取顾客的数据
7     std::vector<Customer> customers;
8     ifstream file;
9     string line;
10    file.open(test_data_file, std::ios::in);
11    while (getline(file, line))
12    {
13        istream iss(line);
14        int id_customer, enter_time, service_time;
15        iss >> id_customer >> enter_time >> service_time;
16        customers.push_back(Customer(id_customer, enter_time, service_time));
17    }
18
19    std::sort(customers.begin(), customers.end(),
20        [](const Customer& a, const Customer& b) {
21            return a.get_enter_time() < b.get_enter_time();
22        });
23
24    std::vector<std::thread> servers;
25    for (int i = 0; i < server_num; i++) {
26        servers.emplace_back(server, i + 1);
27    }
28
29    int last_enter_time = 0;
30    available_servers = server_num; // 初始化
31
32    for (auto& customer : customers) {
33        int interval = customer.get_enter_time() - last_enter_time;
34        if (interval > 0) {
35            std::this_thread::sleep_for(std::chrono::seconds(interval));
36        }
37        last_enter_time = customer.get_enter_time();
38        customer_enter(customer);
39    }
40
41    for (auto& t : servers) {

```

```
42     t.join();
43 }
44
45 return 0;
46 }
```

在主函数中，可以定义银行柜员个数，并完成从文件中读取测例，并得到输出结果的。

2.5 实验测例

测试文件由若干记录组成，记录的字段用空格分开。记录第一个字段是顾客序号，第二字段为顾客进入银行的时间，第三字段是顾客需要服务的时间。在该实验中，设计的测例为：

```
1 1 1 10
2 2 5 2
3 3 6 3
4 4 1 4
5 5 1 5
6 6 1 3
7 7 2 2
8 8 2 7
9 9 3 1
10 10 3 6
```

3 实验结果

在本实验中设置的柜员数量为 2，实验结果如下图：

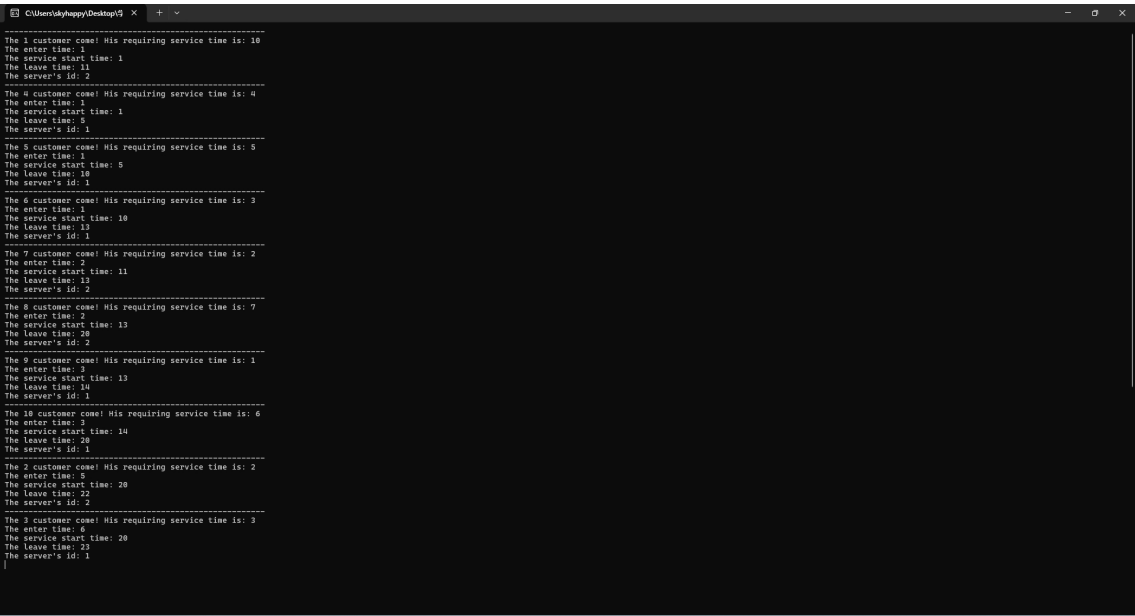


图 1: 实验结果

可以自行验证，该程序结果是正确的。该结果应该是不可复现的，因为抢占顾客的柜员是随机的，不能保证每次都是同一个柜员抢占同一个顾客。

4 思考题

1. 柜员人数和顾客人数对结果分别有什么影响？

柜员人数直接决定系统的并发服务能力，是影响顾客等待时间、队列长度和柜员利用率的关键因素。柜员人数越少，总时间长度越长，人均等待时间也越长；柜员人数越多，总时间长度越短，但柜员的利用率不高。因此，柜员人数很少或者很多在现实中都不是最优的。

顾客人数决定系统的负载压力，其变化会显著影响队列长度和等待时间，甚至导致系统崩溃。如果顾客个数远大于柜员数，或者是同一时刻到来特别多顾客，会导致等待队列大量的积累，使得人均等待时间的延长。

因此，系统的最优状态是柜员人数与顾客人数匹配，像银行的“弹性窗口”机制就是用来动态调整柜员数量的。

2. 实现互斥的方法有哪些？各自有什么特点？效率如何？

实现互斥的方法有很多，可以归类为软件和硬件方法。

1. 软件方法

a. 互斥/自旋锁变量

使用共享变量 `lock`，通过循环检查 `lock` 的值实现互斥，问题是可能违反“忙则等待”原则，并且在极端情况下两个线程同时申请锁变量，可能同时进入临界区。由于忙等待机制，效率极低。

b. 轮转法

使用共享变量 `turn` 指示轮到哪个进程进入，但是由于强制严格轮转进入，会违反“空闲则进”原则。由于强制轮转，效率较低。

c. Peterson 算法

在轮转法的基础上，结合 `turn` 和 `interested` 数组实现互斥，解决了互斥问题，并且不要求严格轮流。但是仍然无法避免忙等待问题，效率也较低。

d. 信号量

对不同的临界资源设定信号量，通过原语 `P (down)` 和 `V (up)` 修改操作系统内核实现的计数器，若计数器为 0 则阻塞当前进程切换到内核，释放时唤醒其他进程。因为涉及到频繁的内核态-用户态切换，并且要更改上下文，使得这一方法的效率适中，但是使得编程复杂度大大提高，同步逻辑分散在代码中，可读性差。

e. 管程

独立出一个线程专门管理信号量，通过编译器生成“入口队列”和“条件变量”，底层通常依赖互斥锁或信号量。

2. 硬件方法

a. 禁止中断

在单处理器系统中，通过屏蔽 CPU 中断（如 x86 的 `cli` 指令）保证临界区执行期间无其他进程抢占，CPU 无法接收到中断信号，也就不会进行进程切换。该方法实现起来难度较低，但是只适合单处理器系统，多处理器系统中其他 CPU 仍然可以执行进程。

b. 硬件指令

CPU 提供原子操作指令（如 `TSL`、`CAS`），保证“读取内存 → 修改内存”操作的原子性。例如：`TSL`、`CAS`。该方法效率较高，是现代多线程互斥的核心实现基础。

5 实验总结

本次实验中，我通过银行柜员问题对进程间同步、互斥问题有了更近一步的了解，明白了信号量的作用，并且将其更加具体地通过 C++ 语言实现出来。在课上主要学习到了一些理论，但是对于具体实现只是一个方法上的指导，在真实实现中往往比预期要困难的多。本次实验用到了以前编写 C++ 程序时从未使用过的库函数，感觉比较陌生，但是在查阅网络资料以及 AI 辅助后也顺利得以解决。在最后做思考题时也对课上所学的互斥、同步的知识做了回顾。

最后，感谢老师和助教的批阅！

6 源代码

```

1  #include <iostream>
2  #include <fstream>
3  #include <sstream>
4  #include <thread>
5  #include <chrono>
6  #include <mutex>
7  #include <condition_variable>
8  #include <vector>
9  #include <queue>
10 #include <algorithm>
11 #include <Windows.h>
12 #include <string>
13
14 using namespace std;
15
16 class Customer {
17 private:
18     int id;           // 顾客序号
19     int num;          // 取号
20     int enter_time;   // 进入银行时间
21     int service_time; // 需要服务的时间
22     int start_time;   // 开始服务时间
23     int leave_time;   // 离开银行时间
24     int server_id;    // 服务柜员号
25
26 public:
27     Customer(int id = 0, int enter_time = 0, int service_time = 0)
28         : id(id), enter_time(enter_time), service_time(service_time),
29           num(-1), start_time(0), leave_time(0), server_id(0) {}
30
31     //private信息接口
32     int get_id() const { return id; }
33     int get_num() const { return num; }
34     int get_enter_time() const { return enter_time; }
35     int get_service_time() const { return service_time; }
36     int get_start_time() const { return start_time; }
37     int get_leave_time() const { return leave_time; }
38     int get_server_id() const { return server_id; }
39
40
41     //设置服务接口
42     void set_id(int idx) { id = idx; }
43     void set_num(int number) { num = number; }
44     void set_enter_time(int t) { enter_time = t; }
45     void set_service_time(int t) { service_time = t; }
46     void set_start_time(int t) { start_time = t; }
47     void set_leave_time(int t) { leave_time = t; }
48     void set_server_id(int id) { server_id = id; }
49 };
50
51 // 全局共享资源（保持不变）
52 int current_num = 0;           // 当前最大取号数（用于生成唯一号码）
53 std::mutex mtx_num;           // 保护取号操作的互斥锁
54 std::queue<Customer> wait_queue; // 等待服务的顾客队列
55 std::mutex mtx_queue;         // 保护队列操作的互斥锁
56
57 // 同步变量
58 int customer_num = 0;         // 等待服务的顾客数量（替代customer_waiting信号量）
59 std::condition_variable cv_customer; // 顾客到达的条件变量（通知柜员）

```

```

60 int available_servers;           //可用柜员数量
61 std::condition_variable cv_server; // 柜员空闲的条件变量（通知顾客）
62
63 // 顾客到达事件处理函数
64 void customer_enter(Customer& customer) {
65     // 取号（互斥操作，确保号码唯一）
66     {
67         std::lock_guard<std::mutex> lock(mtx_num);
68         customer.set_num(++current_num);
69     }
70
71     // 等待可用柜员（无空闲柜员时阻塞）
72     {
73         std::unique_lock<std::mutex> lock(mtx_queue);
74         cv_server.wait(lock, [&] { return available_servers > 0; }); // 等待柜员空闲
75         available_servers--;
76     }
77
78     // 加入等待队列（互斥操作）
79     {
80         std::lock_guard<std::mutex> lock(mtx_queue);
81         wait_queue.push(customer);
82         customer_num++;
83     }
84
85     // 通知有新顾客等待
86     cv_customer.notify_one(); // 唤醒一个柜员
87 }
88
89 void server(int server_id) {
90     thread_local int last_leave_time = 0; // 线程局部变量，记录该柜员上次服务结束时间
91
92     while (true) {
93         // 等待有顾客需要服务
94         Customer customer;
95         {
96             std::unique_lock<std::mutex> lock(mtx_queue);
97             cv_customer.wait(lock, [&] { return customer_num > 0; }); // 等待顾客到达，无顾客时阻塞
98             customer = wait_queue.front();
99             wait_queue.pop();
100             customer_num--;
101         }
102
103         // 计算服务时间
104         int start_time = customer.get_enter_time() > last_leave_time ? customer.get_enter_time() :
            last_leave_time;
105         int leave_time = start_time + customer.get_service_time();
106
107         // 记录服务信息
108         customer.set_start_time(start_time);
109         customer.set_leave_time(leave_time);
110         customer.set_server_id(server_id);
111
112         // 输出结果
113         cout << "-----" << endl;
114         cout << "The " << customer.get_id() << " customer come! His requiring service time is: "
            << customer.get_service_time() << endl;
115         cout << "The enter time: " << customer.get_enter_time() << endl;
116         cout << "The service start time: " << customer.get_start_time() << endl;
117         cout << "The leave time: " << customer.get_leave_time() << endl;
118         cout << "The server's id: " << customer.get_server_id() << endl;
119

```

```

120 // 模拟服务耗时
121 std::this_thread::sleep_for(std::chrono::seconds(customer.get_service_time()));
122
123 // 服务完成, 释放柜员
124 {
125     std::lock_guard<std::mutex> lock(mtx_queue);
126     available_servers++;
127 }
128 cv_server.notify_one(); // 通知顾客有柜员空闲
129
130 // 更新柜员空闲时间
131 last_leave_time = leave_time;
132 }
133 }
134
135 int main() {
136     int server_num = 2; // 柜员数量
137     string test_data_file = "C:/Users/skyhappy/Desktop/test.txt";
138
139
140 // 从文件中读取顾客的数据
141 std::vector<Customer> customers;
142 ifstream file;
143 string line;
144 file.open(test_data_file, std::ios::in);
145 while (getline(file, line))
146 {
147     istringstream iss(line);
148     int id_customer, enter_time, service_time;
149     iss >> id_customer >> enter_time >> service_time;
150     customers.push_back(Customer(id_customer, enter_time, service_time));
151 }
152
153 std::sort(customers.begin(), customers.end(),
154     [](const Customer& a, const Customer& b) {
155         return a.get_enter_time() < b.get_enter_time();
156     });
157
158 std::vector<std::thread> servers;
159 for (int i = 0; i < server_num; i++) {
160     servers.emplace_back(server, i + 1);
161 }
162
163 int last_enter_time = 0;
164 available_servers = server_num; // 初始化
165
166 for (auto& customer : customers) {
167     int interval = customer.get_enter_time() - last_enter_time;
168     if (interval > 0) {
169         std::this_thread::sleep_for(std::chrono::seconds(interval));
170     }
171     last_enter_time = customer.get_enter_time();
172     customer_enter(customer);
173 }
174
175 for (auto& t : servers) {
176     t.join();
177 }
178 return 0;
179 }

```

Listing 1: main.cpp